

Relatório de Testes

HOSPED — Microserviço de Pagamentos (ms-pagamentos)

Versão 1.0

Versão	Data	Alterações	Fonte	Autoria
1.0	03/05/2026	Versão inicial do relatório	PagamentoServiceTest.java PagamentoControllerTest.java	Grupo HOSPED

SUMÁRIO

1. INTRODUÇÃO

2. SISTEMA TESTADO

3. REQUISITOS TESTADOS

4. RELATÓRIO DA EXECUÇÃO DOS CASOS DE TESTE

4.1–4.19. PagamentoServiceTest (21 testes unitários)

4.20–4.24. PagamentoControllerTest (5 testes de integração)

5. RESUMO DA CLASSIFICAÇÃO DOS RESULTADOS OBTIDOS

6. CONCLUSÃO

2

2

2

3

3

13

15

15

1. INTRODUÇÃO

Este documento constitui o Relatório de Testes do microserviço ms-pagamentos, desenvolvido no contexto do projeto acadêmico HOSPED — sistema de gerenciamento hoteleiro baseado em arquitetura de microserviços, conduzido por estudantes do curso de Ciência da Computação da UDESC.

O propósito deste relatório é registrar a execução dos 26 casos de teste planejados para o módulo de pagamentos, distribuídos entre testes unitários da camada de serviço (PagamentoServiceTest — 21 testes) e testes de integração da camada de controle (PagamentoControllerTest — 5 testes).

Os testes foram elaborados utilizando JUnit 5, Mockito, MockitoExtension e MockMvc (@WebMvcTest), cobrindo os principais fluxos de negócio das classes PagamentoService e PagamentoController.

2. SISTEMA TESTADO

O sistema HOSPED é uma aplicação distribuída de gerenciamento hoteleiro composta por múltiplos microserviços. O módulo testado neste relatório é o ms-pagamentos, responsável pelo ciclo de vida dos pagamentos das reservas do estabelecimento.

O microserviço ms-pagamentos implementa as funcionalidades especificadas nos casos de uso CDU0011 (Processar Reserva), CDU0012 (Confirmar Pagamento), CDU0013 (Atualizar Status do Pagamento), CDU0014 (Buscar Pagamento por ID) e CDU0015 (Buscar Pagamentos por Reserva), conforme o Documento de Especificação de Requisitos HOSPED v1.0.

Não houve mudança de escopo entre o planejamento e a execução dos testes. Todos os 26 cenários previstos foram efetivamente executados.

3. REQUISITOS TESTADOS

ID	Descrição do Requisito / Regra	Impacto	Prob.	Prioridade
RF010	O sistema deve permitir ao hóspede escolher entre pagar antecipadamente (Pix/cartão) ou presencialmente no balcão	9	9	81
RF011	O sistema deve processar o pagamento via Pix, cartão de crédito ou débito (pagamento antecipado)	9	9	81
RF012	O sistema deve enviar as informações de pagamento ao e-mail cadastrado do hóspede	5	9	45
RN009	O status da reserva deve seguir o fluxo: Pendente → Ativa → Encerrada (balcão) ou Pendente → Pago → Ativa → Encerrada (antecipado)	9	9	81
RN010	A confirmação do pagamento antecipado é obrigatória para que a reserva avance de Pendente para Pago	9	5	45
RN011	O cancelamento automático por falta de confirmação de pagamento se aplica somente às reservas com pagamento antecipado	9	5	45

4. RELATÓRIO DA EXECUÇÃO DOS CASOS DE TESTE

Esta seção apresenta as evidências da execução de cada caso de teste. Todos os 26 testes foram executados com sucesso, satisfazendo a exigência mínima de 50% de cobertura do projeto HOSPED.

Seção 4.1 a 4.21 — PagamentoServiceTest (Testes Unitários)

4.1. processarReserva — Calcula valor total corretamente

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0011 — Processar Reserva Fluxo Básico passo 3
Roteiro de Teste	RT-PAG-001 — Cálculo do valor total do pagamento
Objetivo do Roteiro	Verificar se o valor total é calculado corretamente: $\text{valorDiaria} \times \text{dias} (\text{checkOut} - \text{checkIn})$
Localização	Classe: PagamentoService Método: processarReserva(ReservaEventoDTO)
Pré-condições	Repositório mockado; evento com $\text{valorDiaria}=\text{R\$}200,00$, $\text{checkIn}=10/06/2025$, $\text{checkOut}=13/06/2025$
Passo a Passo	1. Criar ReservaEventoDTO com os dados acima 2. Mockar save() para retornar pagamento com valor 600 3. Chamar processarReserva(evento) 4. Verificar valor retornado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho de cálculo de valor (CDU0011 passo 3)
Caso de Teste	TC-PAG-001 — Garantir que $\text{valor} = 200 \times 3 = \text{R\$}600,00$
Entradas	$\text{valorDiaria}=200.00$ $\text{checkIn}=2025-06-10$ $\text{checkOut}=2025-06-13$
Resultado Esperado	$\text{resultado.getValor()} == \text{R\$}600,00$
Prioridade	Alta
Pós-condições	Pagamento criado com valor $\text{R\$}600,00$
Resultado Obtido	Teste passou — valor retornado: $\text{R\$}600,00$
Classificação	Validado
Descrição do Resultado	O cálculo de dias e multiplicação pelo valor da diária retornou $\text{R\$}600,00$, satisfazendo RF011.

4.2. processarReserva — Cria pagamento com status PENDENTE

Requisito	RF010
Regra de Negócio	RN009
Caso de Uso	CDU0011 — Processar Reserva Fluxo Básico passo 4
Roteiro de Teste	RT-PAG-002 — Status inicial do pagamento
Objetivo do Roteiro	Verificar se o pagamento é criado com status PENDENTE
Localização	Classe: PagamentoService Método: processarReserva(ReservaEventoDTO)
Pré-condições	Mocks configurados. ReservaEventoDTO com dados válidos.

Passo a Passo	1. Mockar save() para retornar o argumento 2. Chamar processarReserva(evento) 3. Verificar status retornado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — verifica atribuição do status
Caso de Teste	TC-PAG-002 — Status deve ser PENDENTE
Entradas	ReservaEventoDTO com dados válidos
Resultado Esperado	resultado.getStatus() == PENDENTE
Prioridade	Alta
Pós-condições	Pagamento salvo com status PENDENTE
Resultado Obtido	Teste passou — status: PENDENTE
Classificação	Validado
Descrição do Resultado	Status PENDENTE atribuído corretamente, satisfazendo RN009 e CDU0011.

4.3. processarReserva — Persiste pagamento no repositório

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0011 — Processar Reserva Fluxo Básico passo 5
Roteiro de Teste	RT-PAG-003 — Persistência do pagamento
Objetivo do Roteiro	Verificar se repository.save() é chamado ao processar uma reserva
Localização	Classe: PagamentoService Método: processarReserva(ReservaEventoDTO)
Pré-condições	Mock do PagamentoRepository configurado.
Passo a Passo	1. Configurar mock do repository 2. Chamar processarReserva(evento) 3. Verificar se save() foi chamado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — verifica chamada ao repositório
Caso de Teste	TC-PAG-003 — repository.save() deve ser invocado uma vez
Entradas	ReservaEventoDTO com dados válidos
Resultado Esperado	verify(pagamentoRepository).save(any())
Prioridade	Alta
Pós-condições	save() chamado exatamente uma vez
Resultado Obtido	Teste passou — save() verificado
Classificação	Validado
Descrição do Resultado	Repositório chamado corretamente, satisfazendo CDU0011 passo 5.

4.4. processarReserva — Envia e-mail de pagamento pendente

Requisito	RF012
-----------	-------

Regra de Negócio	RN010
Caso de Uso	CDU0011 — Processar Reserva Fluxo Básico passo 6
Roteiro de Teste	RT-PAG-004 — Envio de e-mail de cobrança
Objetivo do Roteiro	Verificar se EmailService é chamado para enviar o e-mail ao hóspede
Localização	Classe: PagamentoService Método: processarReserva(ReservaEventoDTO)
Pré-condições	Mock do EmailService configurado.
Passo a Passo	1. Configurar mock do EmailService 2. Chamar processarReserva(evento) 3. Verificar se enviarEmailPagamentoPendente() foi chamado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — verifica chamada ao serviço de e-mail
Caso de Teste	TC-PAG-004 — enviarEmailPagamentoPendente() deve ser invocado
Entradas	emailHospede = joao@email.com
Resultado Esperado	verify(emailService).enviarEmailPagamentoPendente(any())
Prioridade	Alta
Pós-condições	E-mail enviado ao hóspede com link de confirmação
Resultado Obtido	Teste passou — método de envio verificado
Classificação	Validado
Descrição do Resultado	EmailService chamado corretamente, satisfazendo RF012 e CDU0011 passo 6.

4.5. processarReserva — Define data de expiração válida

Requisito	RF011
Regra de Negócio	RN011
Caso de Uso	CDU0011 — Processar Reserva Fluxo Básico passo 4
Roteiro de Teste	RT-PAG-005 — Data de expiração do pagamento
Objetivo do Roteiro	Verificar se a dataExpiracao é definida e está no futuro (now + 8h)
Localização	Classe: PagamentoService Método: processarReserva(ReservaEventoDTO)
Pré-condições	horasExpiracao = 8 configurado via ReflectionTestUtils.
Passo a Passo	1. Configurar horasExpiracao=8 2. Chamar processarReserva(evento) 3. Verificar dataExpiracao != null e no futuro
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — verifica atribuição da data de expiração
Caso de Teste	TC-PAG-005 — dataExpiracao não nula e posterior ao momento atual
Entradas	horasExpiracao = 8
Resultado Esperado	dataExpiracao != null && isAfter(now)
Prioridade	Média
Pós-condições	Pagamento com dataExpiracao = now + 8h

Resultado Obtido	Teste passou — data de expiração válida
Classificação	Validado
Descrição do Resultado	Data calculada corretamente como now + 8h, satisfazendo RN011.

4.6. confirmarPagamento — Aprova pagamento pendente

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0012 — Confirmar Pagamento Fluxo Básico passo 5
Roteiro de Teste	RT-PAG-006 — Confirmação de pagamento
Objetivo do Roteiro	Verificar se o status é alterado para APROVADO ao confirmar
Localização	Classe: PagamentoService Método: confirmarPagamento(String id)
Pré-condições	Pagamento PENDENTE com dataExpiracao no futuro.
Passo a Passo	1. Mockar findById para retornar pagamento PENDENTE 2. Chamar confirmarPagamento('pag-001') 3. Verificar status
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho principal de confirmação
Caso de Teste	TC-PAG-006 — Status alterado para APROVADO
Entradas	id='pag-001' status=PENDENTE dataExpiracao=now+8h
Resultado Esperado	pagamento.getStatus() == APROVADO
Prioridade	Alta
Pós-condições	Pagamento com status APROVADO persistido
Resultado Obtido	Teste passou — status APROVADO
Classificação	Validado
Descrição do Resultado	Status atualizado para APROVADO, satisfazendo RN009 e CDU0012.

4.7. confirmarPagamento — Publica evento no RabbitMQ

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0012 — Confirmar Pagamento Fluxo Básico passo 6
Roteiro de Teste	RT-PAG-007 — Publicação de evento de confirmação
Objetivo do Roteiro	Verificar se rabbitTemplate.convertAndSend() é chamado após confirmação
Localização	Classe: PagamentoService Método: confirmarPagamento(String id)
Pré-condições	Mock do RabbitTemplate configurado. Pagamento PENDENTE.
Passo a Passo	1. Configurar mocks 2. Chamar confirmarPagamento('pag-001') 3. Verificar convertAndSend()
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca

Critério de Seleção	Fluxo de controle — verifica publicação do evento
Caso de Teste	TC-PAG-007 — convertAndSend() deve ser invocado
Entradas	id = 'pag-001'
Resultado Esperado	verify(rabbitTemplate).convertAndSend(anyString(), anyString(), any())
Prioridade	Alta
Pós-condições	Evento pagamento.processado publicado
Resultado Obtido	Teste passou — convertAndSend() verificado
Classificação	Validado
Descrição do Resultado	Evento publicado corretamente, permitindo que MS Reservas atualize a reserva.

4.8. confirmarPagamento — Envia e-mail de aprovação

Requisito	RF012
Regra de Negócio	RN010
Caso de Uso	CDU0012 — Confirmar Pagamento Fluxo Básico passo 7
Roteiro de Teste	RT-PAG-008 — E-mail de pagamento aprovado
Objetivo do Roteiro	Verificar se e-mail de aprovação é enviado após confirmação
Localização	Classe: PagamentoService Método: confirmarPagamento(String id)
Pré-condições	Mock do EmailService configurado. Pagamento PENDENTE.
Passo a Passo	1. Configurar mocks 2. Chamar confirmarPagamento('pag-001') 3. Verificar enviarEmailPagamentoAprovado()
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — verifica chamada ao e-mail de aprovação
Caso de Teste	TC-PAG-008 — enviarEmailPagamentoAprovado() deve ser invocado
Entradas	id = 'pag-001'
Resultado Esperado	verify(emailService).enviarEmailPagamentoAprovado(any())
Prioridade	Média
Pós-condições	E-mail de confirmação enviado ao hóspede
Resultado Obtido	Teste passou — e-mail de aprovação verificado
Classificação	Validado
Descrição do Resultado	EmailService chamado corretamente, satisfazendo RF012 e CDU0012 passo 7.

4.9. confirmarPagamento — Lança erro se já aprovado

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0012 — Confirmar Pagamento FE02
Roteiro de Teste	RT-PAG-009 — Validação de status duplicado
Objetivo do Roteiro	Verificar se exceção é lançada ao tentar confirmar pagamento já aprovado

Localização	Classe: PagamentoService Método: confirmarPagamento(String id)
Pré-condições	Pagamento com status APROVADO.
Passo a Passo	1. Mockar findById para retornar pagamento APROVADO 2. Chamar confirmarPagamento('pag-001') 3. Verificar exceção
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — status já APROVADO (violação RN009)
Caso de Teste	TC-PAG-009 — ResponseStatusException 400
Entradas	id='pag-001' status=APROVADO
Resultado Esperado	assertThrows(ResponseStatusException.class, ...)
Prioridade	Alta
Pós-condições	Exceção lançada, pagamento não alterado
Resultado Obtido	Teste passou — exceção 400 lançada
Classificação	Validado
Descrição do Resultado	Confirmação duplicada bloqueada, satisfazendo CDU0012 FE02.

4.10. confirmarPagamento — Lança erro se expirado

Requisito	RF011
Regra de Negócio	RN011
Caso de Uso	CDU0012 — Confirmar Pagamento FE03
Roteiro de Teste	RT-PAG-010 — Validação de pagamento expirado
Objetivo do Roteiro	Verificar se exceção é lançada ao confirmar pagamento com dataExpiracao no passado
Localização	Classe: PagamentoService Método: confirmarPagamento(String id)
Pré-condições	Pagamento PENDENTE com dataExpiracao = now - 1h.
Passo a Passo	1. Criar pagamento com dataExpiracao no passado 2. Chamar confirmarPagamento('pag-001') 3. Verificar exceção
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Análise de valor limite — dataExpiracao < now
Caso de Teste	TC-PAG-010 — ResponseStatusException 400 para pagamento expirado
Entradas	id='pag-001' dataExpiracao = now - 1h
Resultado Esperado	assertThrows(ResponseStatusException.class, ...)
Prioridade	Alta
Pós-condições	Exceção lançada, pagamento não confirmado
Resultado Obtido	Teste passou — exceção 400 para expirado
Classificação	Validado
Descrição do Resultado	Verificação de expiração funciona corretamente, satisfazendo CDU0012 FE03.

4.11. confirmarPagamento — Lança erro se não encontrado

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0012 — Confirmar Pagamento FE01
Roteiro de Teste	RT-PAG-011 — Pagamento não encontrado na confirmação
Objetivo do Roteiro	Verificar se exceção 404 é lançada para ID inexistente
Localização	Classe: PagamentoService Método: confirmarPagamento(String id)
Pré-condições	Repository retorna Optional.empty().
Passo a Passo	1. Mockar findById('inexistente') → Optional.empty() 2. Chamar confirmarPagamento('inexistente') 3. Verificar exceção
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — ID inválido
Caso de Teste	TC-PAG-011 — ResponseStatusException 404
Entradas	id = 'inexistente'
Resultado Esperado	assertThrows(ResponseStatusException.class, ...)
Prioridade	Alta
Pós-condições	Exceção 404 lançada
Resultado Obtido	Teste passou — exceção 404 lançada
Classificação	Validado
Descrição do Resultado	NOT_FOUND lançado corretamente para IDs inexistentes, satisfazendo CDU0012 FE01.

4.12. atualizarStatus — Envia e-mail de aprovação

Requisito	RF012
Regra de Negócio	RN009
Caso de Uso	CDU0013 — Atualizar Status Fluxo Básico passo 5
Roteiro de Teste	RT-PAG-012 — E-mail na atualização para APROVADO
Objetivo do Roteiro	Verificar envio de e-mail de aprovação ao atualizar status para APROVADO
Localização	Classe: PagamentoService Método: atualizarStatus(String id, StatusPagamento)
Pré-condições	Pagamento PENDENTE. Mock do EmailService configurado.
Passo a Passo	1. Chamar atualizarStatus('pag-001', APROVADO) 2. Verificar enviarEmailPagamentoAprovado()
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho if(novoStatus == APROVADO)
Caso de Teste	TC-PAG-012 — enviarEmailPagamentoAprovado() deve ser chamado
Entradas	id='pag-001' novoStatus=APROVADO
Resultado Esperado	verify(emailService).enviarEmailPagamentoAprovado(any())

Prioridade	Média
Pós-condições	E-mail de aprovação enviado
Resultado Obtido	Teste passou
Classificação	Validado
Descrição do Resultado	E-mail enviado corretamente, satisfazendo RF012.

4.13. atualizarStatus — Envia e-mail de expiração

Requisito	RF012
Regra de Negócio	RN011
Caso de Uso	CDU0013 — Atualizar Status Fluxo Básico passo 6
Roteiro de Teste	RT-PAG-013 — E-mail na atualização para EXPIRADO
Objetivo do Roteiro	Verificar envio de e-mail de expiração ao atualizar status para EXPIRADO
Localização	Classe: PagamentoService Método: atualizarStatus(String id, StatusPagamento)
Pré-condições	Pagamento existente. Mock do EmailService configurado.
Passo a Passo	1. Chamar atualizarStatus('pag-001', EXPIRADO) 2. Verificar enviarEmailPagamentoExpirado()
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho else if(novoStatus == EXPIRADO)
Caso de Teste	TC-PAG-013 — enviarEmailPagamentoExpirado() deve ser chamado
Entradas	id='pag-001' novoStatus=EXPIRADO
Resultado Esperado	verify(emailService).enviarEmailPagamentoExpirado(any())
Prioridade	Média
Pós-condições	E-mail de expiração enviado
Resultado Obtido	Teste passou
Classificação	Validado
Descrição do Resultado	E-mail de expiração enviado corretamente, satisfazendo RF012 e RN011.

4.14. atualizarStatus — Publica evento no RabbitMQ

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0013 — Atualizar Status Fluxo Básico passo 4
Roteiro de Teste	RT-PAG-014 — Publicação de evento na atualização de status
Objetivo do Roteiro	Verificar se evento é publicado no RabbitMQ ao atualizar status
Localização	Classe: PagamentoService Método: atualizarStatus(String id, StatusPagamento)
Pré-condições	Mock do RabbitTemplate configurado.
Passo a Passo	1. Chamar atualizarStatus('pag-001', APROVADO) 2. Verificar convertAndSend()
Nível de Teste	Unitário

Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — verifica publicação do evento
Caso de Teste	TC-PAG-014 — convertAndSend() deve ser invocado
Entradas	id='pag-001' novoStatus=APROVADO
Resultado Esperado	verify(rabbitTemplate).convertAndSend(anyString(), anyString(), any())
Prioridade	Alta
Pós-condições	Evento publicado na fila do RabbitMQ
Resultado Obtido	Teste passou
Classificação	Validado
Descrição do Resultado	Evento publicado corretamente para qualquer atualização de status.

4.15. atualizarStatus — Lança erro se não encontrado

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0013 — Atualizar Status FE01
Roteiro de Teste	RT-PAG-015 — Erro ao atualizar status inexistente
Objetivo do Roteiro	Verificar se exceção 404 é lançada para ID inexistente na atualização
Localização	Classe: PagamentoService Método: atualizarStatus(String id, StatusPagamento)
Pré-condições	Repository retorna Optional.empty().
Passo a Passo	1. Mockar findById('inexistente') → Optional.empty() 2. Chamar atualizarStatus('inexistente', APROVADO) 3. Verificar exceção
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — ID inválido
Caso de Teste	TC-PAG-015 — ResponseStatusException 404
Entradas	id='inexistente' novoStatus=APROVADO
Resultado Esperado	assertThrows(ResponseStatusException.class, ...)
Prioridade	Alta
Pós-condições	Exceção 404 lançada
Resultado Obtido	Teste passou — exceção 404
Classificação	Validado
Descrição do Resultado	NOT_FOUND lançado corretamente, satisfazendo CDU0013 FE01.

4.16. buscarPorId — Retorna DTO correto

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0014 — Buscar Pagamento por ID Fluxo Básico
Roteiro de Teste	RT-PAG-016 — Busca de pagamento por ID

Objetivo do Roteiro	Verificar se buscarPorId retorna PagamentoResponseDTO com dados corretos
Localização	Classe: PagamentoService Método: buscarPorId(String id)
Pré-condições	Pagamento existente no repositório mock.
Passo a Passo	1. Mockar findById('pag-001') → pagamento 2. Chamar buscarPorId('pag-001') 3. Verificar id, reservald e valor
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho principal de busca
Caso de Teste	TC-PAG-016 — DTO com id='pag-001', reservald='reserva-001', valor=600
Entradas	id = 'pag-001'
Resultado Esperado	dto.getId()=='pag-001' && dto.getReservald()=='reserva-001' && dto.getValor()==600
Prioridade	Média
Pós-condições	DTO retornado com dados corretos
Resultado Obtido	Teste passou — DTO correto
Classificação	Validado
Descrição do Resultado	toResponseDTO mapeia corretamente todos os campos, satisfazendo CDU0014.

4.17. buscarPorId — Lança erro se não encontrado

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0014 — Buscar Pagamento por ID FE01
Roteiro de Teste	RT-PAG-017 — Erro na busca por ID inexistente
Objetivo do Roteiro	Verificar se exceção 404 é lançada ao buscar ID inexistente
Localização	Classe: PagamentoService Método: buscarPorId(String id)
Pré-condições	Repository retorna Optional.empty().
Passo a Passo	1. Mockar findById('inexistente') → Optional.empty() 2. Chamar buscarPorId('inexistente') 3. Verificar exceção
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — ID inválido
Caso de Teste	TC-PAG-017 — ResponseStatusException 404
Entradas	id = 'inexistente'
Resultado Esperado	assertThrows(ResponseStatusException.class, ...)
Prioridade	Média
Pós-condições	Exceção 404 lançada
Resultado Obtido	Teste passou — exceção 404
Classificação	Validado
Descrição do Resultado	NOT_FOUND lançado corretamente, satisfazendo CDU0014 FE01.

4.18. buscarPorReservald — Retorna lista correta

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0015 — Buscar Pagamentos por Reserva Fluxo Básico
Roteiro de Teste	RT-PAG-018 — Busca de pagamentos por reservald
Objetivo do Roteiro	Verificar se buscarPorReservald retorna lista com dados corretos
Localização	Classe: PagamentoService Método: buscarPorReservald(String reservald)
Pré-condições	Repository mock retorna lista com um pagamento para reserva-001.
Passo a Passo	1. Mockar findByReservald('reserva-001') → lista 2. Chamar buscarPorReservald 3. Verificar tamanho e reservald
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — mapeamento da lista
Caso de Teste	TC-PAG-018 — Lista com 1 item e reservald correto
Entradas	reservald = 'reserva-001'
Resultado Esperado	lista.size()==1 && lista.get(0).getReservald()=='reserva-001'
Prioridade	Média
Pós-condições	Lista retornada com dados corretos
Resultado Obtido	Teste passou — lista com 1 item correto
Classificação	Validado
Descrição do Resultado	Lista de DTOs mapeada corretamente, satisfazendo CDU0015.

4.19. buscarPorReservald — Retorna lista vazia

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0015 — Buscar Pagamentos por Reserva FE01
Roteiro de Teste	RT-PAG-019 — Busca por reservald sem pagamentos
Objetivo do Roteiro	Verificar se lista vazia é retornada para reserva sem pagamentos
Localização	Classe: PagamentoService Método: buscarPorReservald(String reservald)
Pré-condições	Repository mock retorna lista vazia.
Passo a Passo	1. Mockar findByReservald('reserva-999') → lista vazia 2. Verificar lista.isEmpty()
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Partição de equivalência — reserva com e sem pagamentos
Caso de Teste	TC-PAG-019 — lista.isEmpty() == true
Entradas	reservald = 'reserva-999'
Resultado Esperado	lista.isEmpty() == true

Prioridade	Baixa
Pós-condições	Lista vazia retornada sem erro
Resultado Obtido	Teste passou — lista vazia sem exceção
Classificação	Validado
Descrição do Resultado	Lista vazia retornada corretamente, satisfazendo CDU0015 FE01.

4.20. confirmarPagamentoPorReserva — Confirma pagamento pendente

Requisito	RF011
Regra de Negócio	RN009 / RN010
Caso de Uso	CDU0012 — Confirmar Pagamento Fluxo Básico
Roteiro de Teste	RT-PAG-020 — Confirmação de pagamento por reservald
Objetivo do Roteiro	Verificar se o método confirmarPagamentoPorReserva localiza o pagamento PENDENTE pela reserva e o aprova
Localização	Classe: PagamentoService Método: confirmarPagamentoPorReserva(String reservald)
Pré-condições	Repository retorna lista com pagamento PENDENTE para reserva-001.
Passo a Passo	1. Mockar findByReservaldAndStatus('reserva-001', PENDENTE) → lista 2. Chamar confirmarPagamentoPorReserva('reserva-001') 3. Verificar status APROVADO e evento RabbitMQ
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — confirmação por reservald
Caso de Teste	TC-PAG-020 — status == APROVADO && convertAndSend() invocado
Entradas	reservald = 'reserva-001' pagamento status = PENDENTE
Resultado Esperado	pagamento.getStatus() == APROVADO && verify(rabbitTemplate).convertAndSend(...)
Prioridade	Alta
Pós-condições	Pagamento aprovado e evento publicado
Resultado Obtido	Teste passou — APROVADO e evento publicado
Classificação	Validado
Descrição do Resultado	Confirmação por reservald funciona corretamente, satisfazendo RN010.

4.21. expirarPagamentosVencidos — Expira pagamentos pendentes vencidos

Requisito	RF011
Regra de Negócio	RN011
Caso de Uso	CDU0013 — Atualizar Status (automático via scheduler)
Roteiro de Teste	RT-PAG-021 — Expiração automática de pagamentos vencidos
Objetivo do Roteiro	Verificar se o scheduler localiza pagamentos PENDENTE com dataExpiracao no passado e os expira
Localização	Classe: PagamentoService Método: expirarPagamentosVencidos()
Pré-condições	Repository retorna lista com pagamento PENDENTE com dataExpiracao = now - 1min.

Passo a Passo	1. Mockar findByStatusAndDataExpiracaoBefore(PENDENTE, any()) → lista 2. Chamar expirarPagamentosVencidos() 3. Verificar retorno == 1, status == EXPIRADO e evento publicado
Nível de Teste	Unitário
Técnica de Teste	Caixa Branca
Critério de Seleção	Fluxo de controle — caminho de expiração automática (RN011)
Caso de Teste	TC-PAG-021 — retorno == 1 && status == EXPIRADO && evento publicado
Entradas	pagamento PENDENTE com dataExpiracao = now - 1min
Resultado Esperado	expirados == 1 && pagamento.getStatus() == EXPIRADO && verify(rabbitTemplate)...
Prioridade	Alta
Pós-condições	Pagamento expirado e evento publicado
Resultado Obtido	Teste passou — 1 pagamento expirado, evento publicado
Classificação	Validado
Descrição do Resultado	Scheduler expira corretamente pagamentos vencidos, satisfazendo RN011.

Seção 4.22 a 4.26 — PagamentoControllerTest (Testes de Integração)

Os testes abaixo utilizam `@WebMvcTest` e `MockMvc` para verificar o comportamento dos endpoints HTTP expostos pelo `PagamentoController`, com o `PagamentoService` mockado via `@MockBean`.

4.22. `deveConsultarStatusIntegracao` — GET `/pagamentos/reserva/{id}/status-integracao` retorna 200

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0014 — Buscar Pagamento por ID (endpoint de integração)
Roteiro de Teste	RT-PAG-022 — Consulta de status de integração por reservald
Objetivo do Roteiro	Verificar se o endpoint retorna HTTP 200 ao consultar o status de integração de uma reserva
Localização	Classe: <code>PagamentoController</code> GET <code>/pagamentos/reserva/{reservald}/status-integracao</code>
Pré-condições	<code>PagamentoService</code> mockado; <code>consultarStatusIntegracao('reserva-001')</code> retorna DTO válido.
Passo a Passo	1. Mockar <code>consultarStatusIntegracao('reserva-001')</code> → <code>StatusPagamentoIntegracaoDTO</code> 2. Executar GET <code>/pagamentos/reserva/reserva-001/status-integracao</code> via <code>MockMvc</code> 3. Verificar status HTTP 200
Nível de Teste	Integração (<code>WebMvcTest</code>)
Técnica de Teste	Caixa Preta
Critério de Seleção	Baseado em casos de uso — endpoint de consulta de status
Caso de Teste	TC-PAG-022 — HTTP 200 OK retornado
Entradas	<code>reservald = 'reserva-001'</code>
Resultado Esperado	<code>status().isOk()</code>
Prioridade	Média
Pós-condições	Resposta 200 com DTO de status
Resultado Obtido	Teste passou — HTTP 200 OK
Classificação	Validado
Descrição do Resultado	Endpoint responde corretamente a GET de consulta de status de integração.

4.23. `deveAtualizarStatus` — PATCH `/pagamentos/{id}/status` retorna 204

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0013 — Atualizar Status do Pagamento
Roteiro de Teste	RT-PAG-023 — Atualização de status via endpoint HTTP
Objetivo do Roteiro	Verificar se o endpoint PATCH retorna HTTP 204 e delega para o service
Localização	Classe: <code>PagamentoController</code> PATCH <code>/pagamentos/{id}/status?status={novoStatus}</code>
Pré-condições	<code>PagamentoService</code> mockado.
Passo a Passo	1. Executar PATCH <code>/pagamentos/pag-001/status?status=APROVADO</code> via <code>MockMvc</code> 2. Verificar HTTP 204 3. Verificar se <code>atualizarStatus('pag-001', APROVADO)</code> foi chamado
Nível de Teste	Integração (<code>WebMvcTest</code>)

Técnica de Teste	Caixa Preta
Critério de Seleção	Baseado em casos de uso — CDU0013 Fluxo Básico
Caso de Teste	TC-PAG-023 — HTTP 204 No Content retornado; service chamado
Entradas	id='pag-001' status=APROVADO
Resultado Esperado	status().isNoContent() && verify(service).atualizarStatus(...)
Prioridade	Alta
Pós-condições	Status atualizado via HTTP
Resultado Obtido	Teste passou — HTTP 204 e service verificado
Classificação	Validado
Descrição do Resultado	Endpoint delega corretamente para o service, satisfazendo CDU0013.

4.24. deveConfirmarPagamentoPorReserva — POST /pagamentos/reserva/{id}/confirmar retorna 204

Requisito	RF011
Regra de Negócio	RN010
Caso de Uso	CDU0012 — Confirmar Pagamento (via reservald)
Roteiro de Teste	RT-PAG-024 — Confirmação de pagamento por reserva via endpoint HTTP
Objetivo do Roteiro	Verificar se o endpoint POST de confirmação por reserva retorna HTTP 204 e delega para o service
Localização	Classe: PagamentoController POST /pagamentos/reserva/{reservald}/confirmar
Pré-condições	PagamentoService mockado.
Passo a Passo	1. Executar POST /pagamentos/reserva/reserva-001/confirmar via MockMvc 2. Verificar HTTP 204 3. Verificar se confirmarPagamentoPorReserva('reserva-001') foi chamado
Nível de Teste	Integração (WebMvcTest)
Técnica de Teste	Caixa Preta
Critério de Seleção	Baseado em casos de uso — CDU0012 confirmação por reservald
Caso de Teste	TC-PAG-024 — HTTP 204 No Content; service chamado
Entradas	reservald = 'reserva-001'
Resultado Esperado	status().isNoContent() && verify(service).confirmarPagamentoPorReserva(...)
Prioridade	Alta
Pós-condições	Confirmação delegada ao service via HTTP
Resultado Obtido	Teste passou — HTTP 204 e service verificado
Classificação	Validado
Descrição do Resultado	Endpoint delega corretamente para confirmarPagamentoPorReserva, satisfazendo CDU0012.

4.25. deveConfirmarPagamentoViaLink — GET /pagamentos/pagar/{id} retorna 200 e mensagem

Requisito	RF011 / RF012
------------------	---------------

Regra de Negócio	RN009 / RN010
Caso de Uso	CDU0012 — Confirmar Pagamento (link do e-mail)
Roteiro de Teste	RT-PAG-025 — Confirmação de pagamento via link do e-mail
Objetivo do Roteiro	Verificar se o endpoint GET de confirmação via link retorna HTTP 200 e a mensagem de sucesso
Localização	Classe: PagamentoController GET /pagamentos/pagar/{id}
Pré-condições	PagamentoService mockado.
Passo a Passo	1. Executar GET /pagamentos/pagar/pag-001 via MockMvc 2. Verificar HTTP 200 3. Verificar mensagem 'Pagamento confirmado! Obrigado, sua reserva está garantida.' 4. Verificar se confirmarPagamento('pag-001') foi chamado
Nível de Teste	Integração (WebMvcTest)
Técnica de Teste	Caixa Preta
Critério de Seleção	Baseado em casos de uso — CDU0012 acionado pelo botão do e-mail
Caso de Teste	TC-PAG-025 — HTTP 200 OK com mensagem de confirmação
Entradas	id = 'pag-001'
Resultado Esperado	status().isOk() && content().string('Pagamento confirmado! ...')
Prioridade	Alta
Pós-condições	Mensagem de confirmação retornada ao hóspede
Resultado Obtido	Teste passou — HTTP 200 e mensagem correta
Classificação	Validado
Descrição do Resultado	Endpoint do botão do e-mail funciona corretamente, satisfazendo CDU0012 e RF012.

4.26. deveSimularReserva — POST /pagamentos/teste/simular-reserva retorna 200

Requisito	RF011
Regra de Negócio	RN009
Caso de Uso	CDU0011 — Processar Reserva (endpoint de teste)
Roteiro de Teste	RT-PAG-026 — Simulação de reserva via endpoint de teste
Objetivo do Roteiro	Verificar se o endpoint de simulação retorna HTTP 200, mensagem correta e delega para processarReserva
Localização	Classe: PagamentoController POST /pagamentos/teste/simular-reserva
Pré-condições	PagamentoService mockado.
Passo a Passo	1. Criar ReservaEventoDTO com reservId='reserva-001' 2. Executar POST /pagamentos/teste/simular-reserva com body JSON 3. Verificar HTTP 200 e mensagem 'Reserva simulada com sucesso!' 4. Verificar se processarReserva(any()) foi chamado
Nível de Teste	Integração (WebMvcTest)
Técnica de Teste	Caixa Preta
Critério de Seleção	Baseado em casos de uso — CDU0011 via endpoint de simulação
Caso de Teste	TC-PAG-026 — HTTP 200 OK com mensagem 'Reserva simulada com sucesso!'
Entradas	body: ReservaEventoDTO com reservId='reserva-001'
Resultado Esperado	status().isOk() && content().string('Reserva simulada com sucesso!')

Prioridade	Média
Pós-condições	Simulação processada com sucesso
Resultado Obtido	Teste passou — HTTP 200 e mensagem correta
Classificação	Validado
Descrição do Resultado	Endpoint de simulação funciona corretamente para testes de desenvolvimento.

5. RESUMO DA CLASSIFICAÇÃO DOS RESULTADOS OBTIDOS

A tabela abaixo apresenta o consolidado dos resultados obtidos na execução dos 26 casos de teste do ms-pagamentos, cobrindo os requisitos RF010, RF011, RF012 e as regras de negócio RN009, RN010 e RN011 do documento HOSPED v1.0.

Nível de Testes	#Total	#Validados	#Bugs (urgente, alto, médio ou baixo)	#Melhorias (necessária ou desejável)
Testes Unitários (Service)	21	21	0, 0, 0, 0	0, 0
Testes de Integração (Controller)	5	5	0, 0, 0, 0	0, 0
Testes de Sistema	—	—	—	—
Testes de Aceitação	—	—	—	—
TOTAL	26	26	0, 0, 0, 0	0, 0

6. CONCLUSÃO

Este relatório documentou a execução de 26 casos de teste para o microserviço ms-pagamentos do sistema HOSPED: 21 testes unitários (PagamentoServiceTest) e 5 testes de integração (PagamentoControllerTest). Todos foram classificados como Validados, sem identificação de bugs ou melhorias necessárias.

Os testes cobrem os requisitos RF010, RF011 e RF012 e as regras de negócio RN009, RN010 e RN011, diretamente rastreáveis aos casos de uso CDU0011, CDU0012, CDU0013, CDU0014 e CDU0015 do Documento de Especificação de Requisitos HOSPED v1.0.

A adoção de Mockito e MockMvc permitiu isolar completamente as camadas de serviço e controle, assegurando que os testes validem exclusivamente a lógica de negócio sem dependência de banco de dados, RabbitMQ ou serviço de e-mail. A experiência reforçou a importância da rastreabilidade entre testes e requisitos: vincular cada caso de teste ao RF, RN e CDU corretos facilita a identificação de falhas de cobertura e documenta com precisão quais regras de negócio estão sendo verificadas — prática essencial em sistemas distribuídos como o HOSPED.